

Лабораторная работа № 13

Отчет

Гайдук Софья Сергеевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Контрольные вопросы	15
4	Выводы	17
	Список литературы	18

Список иллюстраций

2.1	image1	6
2.2	image2	6
2.3	image3	7
2.4	image4	8
2.5	image5	8
2.6	image6	9
2.7	image7	9
2.8	image8	10
2.9	image9	10
2.10	image10	10
2.11	image11	11
2.12	image12	12
2.13	image13	13
2.14	image14	14
2.15	image15	14
2.16	image16	14
2.17	image17	14

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

2 Выполнение лабораторной работы

Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r` (рис. 2.1, рис. 2.2, рис. 2.3, рис. 2.4, рис. 2.5).

```
ssgayjduk1@ssgayjduk1:~$ mkdir lab13
ssgayjduk1@ssgayjduk1:~$ cd lab13
ssgayjduk1@ssgayjduk1:~/lab13$ touch text.sh
ssgayjduk1@ssgayjduk1:~/lab13$ touch n1.sh
```

Рисунок 2.1: image1

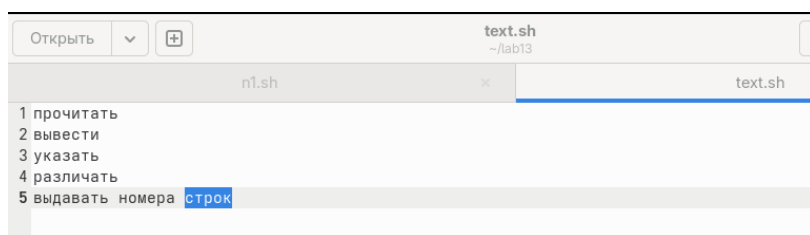


Рисунок 2.2: image2



```
1 #!/bin/bash
2
3 while getopts ":i:o:p:Cn" opt; do
4     case ${opt} in
5         i )
6             inputfile=$OPTARG
7             ;;
8         o )
9             outputfile=$OPTARG
10            ;;
11        p )
12            pattern=$OPTARG
13            ;;
14        C )
15            case_sensitive=true
16            ;;
17        n )
18            line_numbers=true
19            ;;
20        \? )
21            echo "Неверный параметр: $OPTARG" 1>&2
22            exit 1
23            ;;
24        : )
25            echo "Отсутствуют значения параметра: $OPTARG" 1>&2
26            exit 1
27            ;;
28    esac
29 done
30
31 if [ -z "$pattern" ]; then
32     echo "Нет шаблона поиска" 1>&2
33     exit 1
34 fi
35
36
37 if [ -z "$inputfile" ]; then
```

Рисунок 2.3: image3

```

37 if [ -z "$inputfile" ]; then
38     echo "Нет входного файла" 1>&2
39     exit 1
40 fi
41
42
43 if [ "$case_sensitive" = true ]; then
44     grep_options+=" -i"
45 fi
46
47
48 if [ "$line_numbers" = true ]; then
49     grep_options+=" -n"
50 fi
51
52 grep $grep_options "$pattern" "$inputfile"
53
54 if [ ! -z "$outputfile" ]; then
55     grep $grep_options "$pattern" "$inputfile" > $outputfile
56 fi
57
58

```

Рисунок 2.4: image4

```

ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh
Нет шаблона поиска
ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh -p "прочитать"
Нет входного файла
ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh -p "прочитать" -i text
grep: text: Нет такого файла или каталога
ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh -p "прочитать" -i text.sh
прочитать
ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh -p "строК" -i text.sh -n
5:выдавать номера строк
ssgayjduk1@ssgayjduk1:~/lab13$ ./n1.sh -p "строК" -i text.sh -n -C
5:выдавать номера строк
ssgayjduk1@ssgayjduk1:~/lab13$

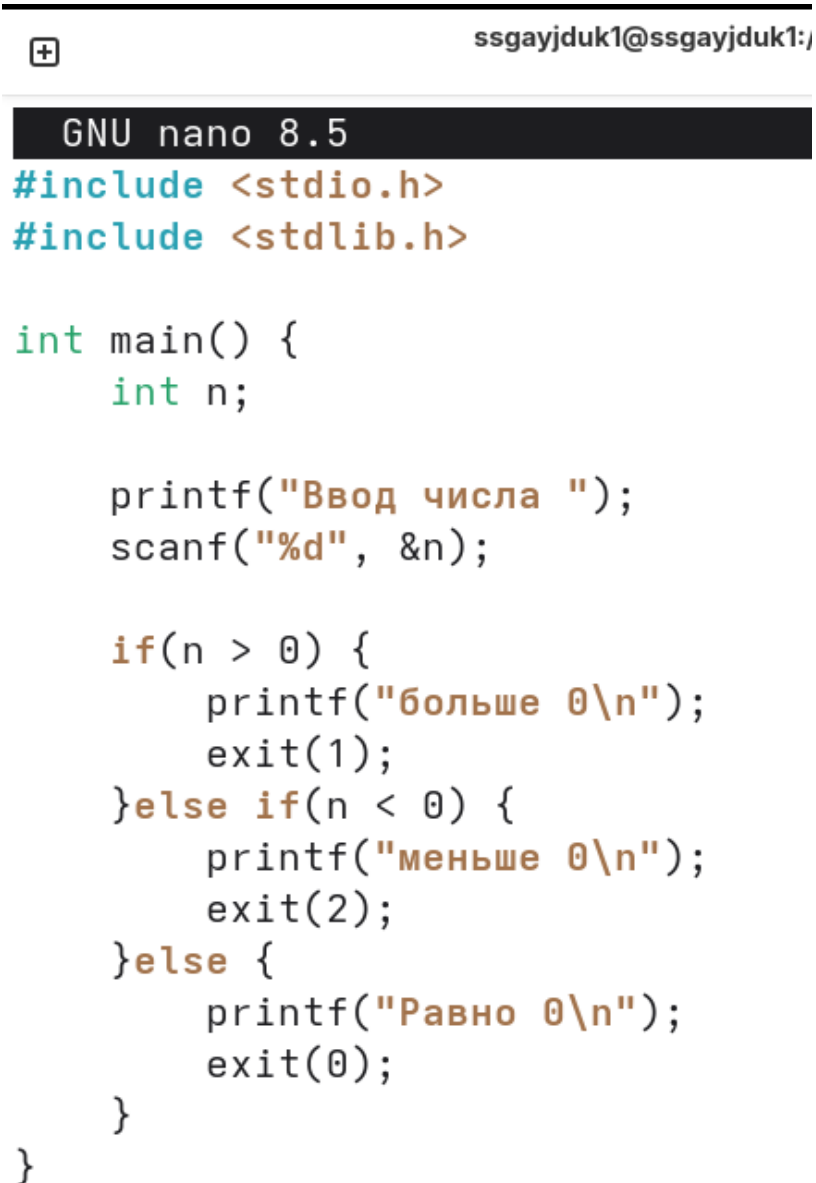
```

Рисунок 2.5: image5

Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `?`, выдать сообщение о том, какое число было введено (рис. 2.6, рис. 2.7, рис. 2.8, рис. 2.9, рис. 2.10, рис. 2.11).


```
ssgayjduk1@ssgayjduk1:~/lab13$ touch n2.c
ssgayjduk1@ssgayjduk1:~/lab13$ touch n2.sh
ssgayjduk1@ssgayjduk1:~/lab13$ chmod +x n2.sh
ssgayjduk1@ssgayjduk1:~/lab13$ nano n2.c
```

Рисунок 2.6: image6



```
ssgayjduk1@ssgayjduk1:/
GNU nano 8.5
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;

    printf("Ввод числа ");
    scanf("%d", &n);

    if(n > 0) {
        printf("больше 0\n");
        exit(1);
    }else if(n < 0) {
        printf("меньше 0\n");
        exit(2);
    }else {
        printf("Равно 0\n");
        exit(0);
    }
}
```

Рисунок 2.7: image7

```
ssgayjduk1@ssgayjduk1:~/lab13$ gcc n2.c -o n2
```

Рисунок 2.8: image8

```
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2
Ввод числа 10
больше 0
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2
Ввод числа -1
меньше 0
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2
Ввод числа 0
Равно 0
ssgayjduk1@ssgayjduk1:~/lab13$ █
```

Рисунок 2.9: image9

```
#!/bin/bash

./n2|

case $? in
    0)
        echo "равно нулю";;
    1)
        echo "больше нуля";;
    2)
        echo "меньше нуля";;
esac
```

Рисунок 2.10: image10

```
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2.sh
Ввод числа 10
больше 0
больше нуля
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2.sh
Ввод числа -1
меньше 0
меньше нуля
ssgayjduk1@ssgayjduk1:~/lab13$ ./n2.sh
Ввод числа 0
Равно 0
равно нулю
ssgayjduk1@ssgayjduk1:~/lab13$ █
```

Рисунок 2.11: image11

Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют) (рис. 2.12, рис. 2.13).

```
#!/bin/bash

add_files() {
    local count=$1
    for ((i=1; i <= $count; i++)); do
        touch "$i.tmp"
        echo "создан файл $i.tmp"
    done
}

del_files() {
    local count=$1
    for ((i=1; i <= $count; i++)); do
        if [ -e "$i.tmp" ]; then
            rm "$i.tmp"
            echo "удален файл $i.tmp"
        fi
    done
}

if [ $# -eq 0 ]; then
    echo "нет количества файлов для создания"
    exit 1
fi

action=$1

case $action in
    create)
        add_files $2
        ;;
    delete)
        del_files $2
        ;;
    *)
        echo "введите create (создать) или delete (удалить) файлы"
        exit 1
        ;;
esac
```

Рисунок 2.12: image12

```
ssgayjduk1@ssgayjduk1:~/lab13$ touch n3.sh
ssgayjduk1@ssgayjduk1:~/lab13$ ./n3.sh
нет количества файлов для создания
ssgayjduk1@ssgayjduk1:~/lab13$ ./n3.sh 7
введите create (создать) или delete (удалить) файлы
ssgayjduk1@ssgayjduk1:~/lab13$ ./n3.sh create 7
создан файл 1.tmp
создан файл 2.tmp
создан файл 3.tmp
создан файл 4.tmp
создан файл 5.tmp
создан файл 6.tmp
создан файл 7.tmp
ssgayjduk1@ssgayjduk1:~/lab13$ ./n3.sh delete 7
удален файл 1.tmp
удален файл 2.tmp
удален файл 3.tmp
удален файл 4.tmp
удален файл 5.tmp
удален файл 6.tmp
удален файл 7.tmp
ssgayjduk1@ssgayjduk1:~/lab13$ touch n4.sh
ssgayjduk1@ssgayjduk1:~/lab13$ chmod +x n4.sh
ssgayjduk1@ssgayjduk1:~/lab13$
```

Рисунок 2.13: image13

Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`) (рис. 2.14, рис. 2.15, рис. 2.16, рис. 2.17).

```
Открыть ▾  n4.sh  
~/lab13  
#!/bin/bash  
  
directory=$1  
output_ar="archive.tar.gz"  
day=7  
  
if [ -z "$directory" ]; then  
    echo "указать директорию"  
    exit 1  
fi  
  
if [ ! -d "$directory" ]; then  
    echo "нет такой директории"  
    exit 1  
fi  
  
find "$directory" -type f -mtime -$day -print0 | tar --null -czf "$output_ar" --files-from -  
echo "Архив создан $output_ar"
```

Рисунок 2.14: image14

```
ssgayjduk1@ssgayjduk1:~/lab13$ ./n4.sh +  
нет такой директории
```

Рисунок 2.15: image15

```
ssgayjduk1@ssgayjduk1:~/lab13$ ./n4.sh .  
Архив создан archive.tar.gz
```

Рисунок 2.16: image16

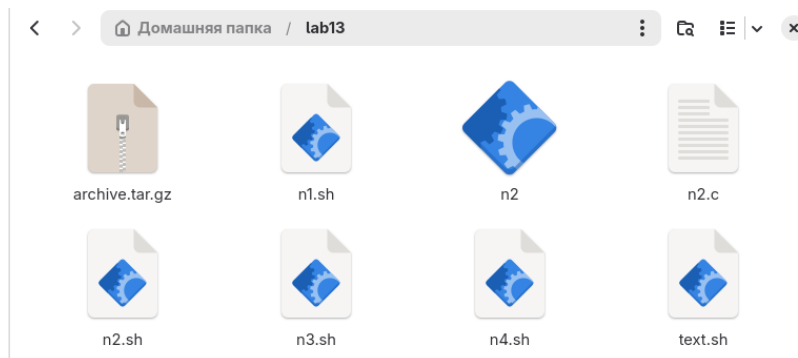


Рисунок 2.17: image17

3 Контрольные вопросы

1. Каково предназначение команды `getopts`?

- `getopts` — разбирает параметры командной строки (например, `-i file -o out`) в скриптах `bash`.

2. Какое отношение метасимволы имеют к генерации имён файлов?

- Метасимволы (`*`, `?`, `[]`) используются для подстановки имён файлов (glob pattern), заменяясь списком подходящих файлов.

3. Какие операторы управления действиями вы знаете?

- Операторы управления действиями — `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер).

4. Какие операторы используются для прерывания цикла?

- Операторы прерывания цикла — `break` (полный выход), `continue` (переход к следующей итерации).

5. Для чего нужны команды `false` и `true`?

- `true` — всегда возвращает код 0 (успех). `false` — всегда возвращает код 1 (ошибка). Используются для бесконечных циклов или проверок.

6. Что означает строка `if test -f mans/i.$s`, встречаемая в командном файле?

- `if test -f mans/i.$s` — проверяет, существует ли файл с именем, составленным из переменных и расширений (например, `man1/prog.c`).

7. Объясните различия между конструкциями `while` и `until`.

- `while` — выполняет цикл, пока условие истинно. `until` — выполняет цикл, пока условие ложно (до тех пор, пока не станет истинным).

4 Выводы

Изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

Список литературы

1.Kulyabov. Лабораторная работа № 13. Программирование в командном процессе ОС UNIX. Ветвления и циклы. RUDN